CS 39006: Computer Networks Laboratory

POSIX Network Programming: TCP Sockets

Department of Computer Science and Engineering

INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

Abhijnan Chakraborty
abhijnan@cse.iitkgp.ac.in

Sandip Chakraborty
sandipc@cse.iitkgp.ac.in

TCP Sockets

Key Characteristics

- Connection-oriented (3-way handshake)
- Reliable, ordered byte stream
- Automatic retransmission, flow control, congestion control

POSIX Type:

`SOCK_STREAM`

TCP Socket API (High-Level)

Core System Calls

- `socket()`
- `bind()`
- `listen()`
- `accept()`
- `connect()`
- `read()`, `write()` or `recv()`, `send()`
- `close()`

TCP Server Architecture

Typical Flow

1. `socket()`
2. `bind()`
3. `listen()`
4. `accept()` → blocks until client connects
5. `read()/write()`
6. `close()`

TCP Client Architecture

Typical Flow

1. `socket()`
2. `connect()` to server
3. `read()/write()`
4. `close()`

Creating a TCP Socket

Header Files

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>
```

Creating a socket

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

Creating a TCP Socket

Creating a socket

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

Parameters

- `AF_INET` → IPv4
- `SOCK_STREAM` → TCP
- 0 → default protocol (TCP)

Error Handling

```
if (sockfd < 0) perror("socket failed");
```

Socket Address Structure (Same as UDP)

```
struct sockaddr_in server_addr;
```

Fields:

```
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
server_addr.sin_addr.s_addr = INADDR_ANY;
```

Binding (Same as UDP) and Listening

```
bind(sockfd,
     (struct sockaddr *)&server_addr,
     sizeof(server_addr));
```

Checking bind:

```
if (bind(...) < 0) perror("bind failed");
```

Start Listening

```
listen(sockfd, 5);    // backlog = 5
```

Binding (Same as UDP) and Listening

When a TCP client calls `connect()`, the connection undergoes the **3-way handshake**.

Until the handshake completes and the server calls `accept()`, the connection sits in a kernel-managed pending queue.

The **`backlog`** value determines the maximum number of incoming connection requests that can wait in this queue.

Start listening

```
listen(sockfd, 5);    // backlog = 5
```

Binding (Same as UDP) and Listening

When a TCP client calls `connect()`, the connection undergoes the **3-way handshake**.

Until the handshake completes and the server calls `accept()`, the connection sits in a kernel-managed pending queue.

The **`backlog`** value determines the maximum number of incoming connection requests that can wait in this queue.

What happens if this queue is full?

```
listen(sockfd, 5);    // back
```

Kernel Queue Management

Modern Linux maintains two queues:

1. **SYN Queue (Half-open queue)**: Contains connections that have not yet completed the 3-way handshake (after receiving SYN).
2. **Accept Queue (Completed queue):** Contains connections fully established, but the server has not yet called `accept()`.

The `backlog` value limits the size of the ACCEPT queue, not the SYN queue (which may have its own system limits).

Kernel Queue Management

Modern Linux maintains two queues:

1.

2.

Th
(w

Backlog Is Only a Hint

The OS may cap it internally. On Linux, the maximum is `/proc/sys/net/core/somaxconn` (often 128 or 4096)

The actual backlog used becomes:
`clamp(adjusted_backlog , 1, somaxconn)`

Thus calling listen(sockfd, 10000) doesn't guarantee a queue of 10,000.

Accepting Connection

```
int client_sock;
struct sockaddr_in client_addr;
socklen_t len = sizeof(client_addr);

client_sock = accept(sockfd,
                     (struct sockaddr *)&client_addr,
                     &len);
```

Note:

1. `accept()` blocks until a client requests connection.
2. Returns a new socket for data transfer.

TCP Server: Receiving & Sending

Reading and Writing

```
char buffer[1024];
int n = read(client_sock, buffer, sizeof(buffer));
write(client_sock, "Hello Client", 12);
```

or using `recv() / send()`

Minimal TCP Server

```
int sockfd, client_sock;
struct sockaddr_in server_addr, client_addr;
char buffer[1024];
socklen_t len;

sockfd = socket(AF_INET, SOCK_STREAM, 0);

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
server_addr.sin_addr.s_addr = INADDR_ANY;
```

Minimal TCP Server

```
bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr));
listen(sockfd, 5);

len = sizeof(client_addr);
client_sock = accept(sockfd, (struct sockaddr*) &client_addr,
&len);

read(client_sock, buffer, sizeof(buffer));
write(client_sock, "ACK", 3);

close(client_sock);
close(sockfd);
```

TCP Client Basics

```
int sockfd;
struct sockaddr_in server_addr;
sockfd = socket(AF_INET, SOCK_STREAM, 0);

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
inet_pton(AF_INET, "127.0.0.1", &server_addr.sin_addr);
connect(sockfd, (struct sockaddr *)&server_addr,
sizeof(server_addr));

write(sockfd, "Hello Server", 12);
close(sockfd);
```

Common Pitfalls

- Forgetting `listen()` before `accept()`
- Not handling partial `read()`/`write()`
  - **Assuming message boundaries (TCP is a stream)**
- Not closing sockets (descriptor leak)

Food for Thought

1. What will happen if you make a `read()/write()` call on a TCP socket without doing a `connect()` from the client?
2. What will happen if you make a `connect()` call on a socket where the type is `SOCK_DGRAM`?
3. What happens if you call `accept()` before calling `listen()`?
4. What happens if the server calls `accept()` twice even though only one client connects?
5. What does the server see if a client dies (crash, power loss) without calling `close()`?
6. What happens if the server closed the connection while the client is still writing?
7. What happens if you set backlog = 0?
8. What happens if two different processes try to bind the same (IP, port)?